

Transformations de type reducer

Module 5 — groupBy, agg, Window functions et jointures

Objectifs pédagogiques

- Comprendre le rôle et le coût des opérations de shuffle réseau
- Appliquer les transformations `reduceByKey()`, `groupByKey()` et `aggregateByKey()` sur des RDD
- Réaliser des agrégations complexes sur des DataFrames avec `groupBy()` et `agg()`
- Utiliser les fonctions de fenêtre (Window functions) pour des calculs avancés
- Réaliser et optimiser des jointures entre DataFrames

1. Le reducer : principe et coût réseau

Reducer vs Mapper — le rôle du shuffle

Contrairement aux mappers, les reducers nécessitent de rassembler des données de différentes partitions

MAPPER

*Chaque partition traitée
indépendamment*

- Partition 1 → traitée sur Exec 1
- Partition 2 → traitée sur Exec 2
- Partition 3 → traitée sur Exec 3
- Aucun transfert réseau

SHUFFLE

(réseau)

REDUCER

*Données redistribuées
par clé via le réseau*

```
P1: [A:1, B:3, A:2]
P2: [B:1, A:4, C:2] →shuffle→
P3: [C:1, B:2, C:3]
```

```
Partition A: [1,2,4] → A:7
```

```
Partition B: [3,1,2] → B:6
```

```
Partition C: [2,1,3] → C:6
```

Opérations déclenchant un shuffle : `groupBy()` • `reduceByKey()` • `join()` • `distinct()` • `repartition()`
• `sortBy()` • `groupByKey()` • `cogroup()` • `intersection()`

Le shuffle — opération la plus coûteuse

3 étapes coûteuses : écriture disque → transfert réseau → lecture et désérialisation

Étape 1

Écriture sur disque

Chaque Executor sérialise et écrit les données à transférer vers les autres nœuds

Étape 2

Transfert réseau

Les données sont copiées via le réseau vers les Executors destinataires

Étape 3

Lecture + désérialisation

Les Executors destinataires reconstituent les données en mémoire pour le calcul

Stratégies pour minimiser le coût des shuffles

```
# ❌ Filtrer APRÈS groupBy → shuffle sur les données non filtrées
df.groupBy("ville").count().filter(F.col("count") > 10)
# ✅ Filtrer AVANT groupBy → moins de données à shuffler
df.filter(F.col("montant") > 0).groupBy("ville").count().filter(F.col("count") > 10)
# ✅ Broadcast join pour les petites tables (évite le shuffle total)
df_large.join(broadcast(df_petite), "id")
# ✅ Cacher un résultat de shuffle souvent réutilisé
df_groupe = df.groupBy("ville").agg(...).cache() ; df_groupe.count()
```


2. Reducers sur les RDD

reduce() et reduceByKey()

reduce() → valeur scalaire unique · reduceByKey() → agrégation par clé sur un RDD de paires

```
# reduce() — action (scalaire unique)
rdd = sc.parallelize(range(1, 11))
rdd.reduce(lambda a,b: a+b)      # → 55
rdd.reduce(lambda a,b: a if a>b else b) # → 10
```

```
# ⚠ La fonction doit être :
# ✅ commutative : f(a,b) == f(b,a)
# ✅ associative : f(f(a,b),c) == f(a,f(b,c))
# ❌ soustraction : ni l'un ni l'autre !
```

```
# reduceByKey() — transformation (RDD de paires)
rdd_v = sc.parallelize([
    ("Paris",1500.), ("Lyon",800.),
    ("Paris",2200.), ("Lyon",1100.)])
```

```
rdd_v.reduceByKey(lambda a,b: a+b).collect()
# [('Paris',3700.), ('Lyon',1900.)]
```

```
# Word count : le pattern classique
rdd_mots.map(lambda m: (m,1))\
    .reduceByKey(
        lambda a,b: a+b)
```

reduceByKey() — réduction locale AVANT shuffle → beaucoup moins de données transférées

```
groupByKey() : P1:[(A,1),(A,2)] —réseau→ [(A,1),(A,2),(A,4)] → A:7 (toutes les valeurs)
reduceByKey() : P1:[(A,1),(A,2)] —local→ (A,3) —réseau→ (A,3)+(A,4) → A:7 (sous-totaux)
```

→ reduceByKey transfère BEAUCOUP moins de données sur le réseau

groupByKey()

À éviter sur gros volumes

Création du RDD

```
rdd_notes = sc.parallelize([
    ("Alice",15),("Bob",12),("Alice",18),
    ("Bob",16),("Alice",14)])

for etudiant, notes in rdd_notes.groupByKey().collect():
    liste = list(notes)
    print(f"{etudiant}: moy={sum(liste)/len(liste):.1f}")
```

Groupement sur les prénoms
(groupByKey)

⚠ **Risque** : si une clé a des millions de valeurs → OutOfMemoryError. Préférer aggregateByKey().

aggregateByKey()

version la plus générale

seq_op prend chaque
notecrée une structure de
données intermédiaire (V):
(0,0) & 12 -> (12,1)

comb_op prend chaque V et
modifie l'accumulateur en
fonction :
(12, 1) & (10,1) -> (22,2)

Valeurs initiales

```
rdd = sc.parallelize([("alice", 12), ("bob", 13), ("alice", 10)])  
zero = (0.0, 0) # (somme, count)
```

```
def seq_op(acc, v):  
    return (acc[0]+v, acc[1]+1)
```

```
def comb_op(a, b):  
    return (a[0]+b[0], a[1]+b[1])
```

3 arguments : zeroValue, seqOp, combOp

```
stats = rdd.aggregateByKey(zero, seq_op, comb_op)  
moyennes = stats.mapValues(lambda s: s[0]/s[1])
```

→ [('Alice',15.67),('Bob',14.0)]

combineByKey() — variante encore plus générale d'aggregateByKey() avec contrôle total sur la création du combineur initial. Utile pour calculer (min, max, somme, count) en une seule passe sur le réseau.

sortByKey() & sortBy()

```
rdd_pair.sortByKey(ascending=False).collect()  
word_count.sortBy(lambda x: -x[1]).take(10) # Top 10 mots par fréquence décroissante
```

3. Reducers sur les DataFrames

groupBy() + agg() — Agrégations sur DataFrame

Combiner plusieurs agrégations en une seule passe — recommandé pour les performances

```
# Agrégations multiples en une seule passe ← recommandé
df.groupBy("ville").agg(
  F.count("*").alias("nb_vendeurs"),
  F.sum("montant").alias("ca_total"),
  F.avg("montant").alias("ca_moyen"),
  F.min("montant").alias("ca_min"), F.max("montant").alias("ca_max"),
  F.stddev("montant").alias("ecart_type"),
  F.collect_list("nom").alias("vendeurs")
).orderBy(F.col("ca_total").desc()).show()
```

Groupement multi-colonnes

```
df.groupBy("ville", "annee").agg(
  F.sum("montant").alias("ca"),
  F.count("*").alias("n")
).orderBy(
  "ville", "annee").show()
```

Percentiles et médiane

```
df.groupBy("ville").agg(
  F.percentile_approx("montant", 0.5),
  F.percentile_approx("montant", [0.25, 0.75])
).show()
```

Fonctions d'agrégation natives — référence

Fonction	Description
F.count('*') / F.count(col)	Nb lignes (avec nulls) / Nb valeurs non-null
F.countDistinct(col)	Nombre de valeurs distinctes
F.sum(col) / F.avg(col)	Somme / Moyenne (ignore les nulls)
F.min(col) / F.max(col)	Minimum / Maximum
F.stddev(col) / F.stddev_pop(col)	Écart-type échantillon / population
F.variance(col)	Variance
F.skewness(col) / F.kurtosis(col)	Asymétrie / Aplatissement
F.corr('col1','col2')	Corrélation de Pearson
F.percentile_approx(col, 0.5)	Médiane approchée (ou [0.25,0.75] pour Q1/Q3)
F.collect_list(col)	Liste de valeurs (avec doublons)
F.collect_set(col)	Ensemble de valeurs (sans doublons)
F.first(col, ignorenulls=True)	Premier élément non-null

pivot() — Tableau croisé dynamique

Transforme des valeurs de lignes en colonnes — équivalent d'un tableau croisé Excel

```
# Pivot : une colonne par année
df_ventes\
  .groupBy("ville")\
  .pivot("annee", [2022, 2023, 2024])\
  .sum("ca")\
  .show()

# Unpivot (inverse — Spark 3.4+)
df_pivot.unpivot(
  "ville", ["2022", "2023", "2024"],

  "annee", "ca").show()
```

Résultat du pivot :

ville	2022	2023	2024
Paris	5000	6200	7100
Lyon	3100	3400	3800
Nantes	1800	2100	2400

Conseil performance : toujours spécifier la liste des valeurs pivot explicitement (ex: [2022,2023,2024]). Sans cela, Spark doit d'abord scanner toutes les données pour découvrir les valeurs distinctes → un shuffle supplémentaire.

Agrégations spéciales sur collections

```
F.collect_list("nom")    # → ['Alice', 'Bob', 'Claire'] (avec doublons)
F.array_join(F.collect_list("nom"), ", ")    # → 'Alice, Bob, Claire'
```

4. Les fonctions de fenêtre (Window functions)

Window functions — Calculs sans réduction du DataFrame

Calcul sur une fenêtre de lignes voisines — le DataFrame garde son nombre de lignes

Cas d'usage :

Définir une fenêtre

Rang dans un groupe

ex : 3ème vendeur de Paris par CA

Cumul progressif

ex : CA cumulé mois par mois

Moyenne glissante

ex : Moyenne sur les 3 derniers mois

Comparaison ligne précédente

ex : Évolution vs mois précédent

```
from pyspark.sql.window import Window
```

```
# Partitionnée + triée
```

```
w = Window.partitionBy("ville")\
           .orderBy(F.col(
               "montant").desc())
```

```
# Fenêtre glissante : N-2 à N
```

```
w_gliss = w.rowsBetween(-2, 0)
```

```
# Cumulée depuis le début
```

```
w_cumul = w.rowsBetween(
    Window.unboundedPreceding,
    Window.currentRow)
```

```
# Délimitée en valeur
```

```
w_range = w.rangeBetween(-7, 0) # ±7 unités
```

Différence rowsBetween vs rangeBetween : rowsBetween compte les lignes physiques (-2 = 2 lignes avant) — rangeBetween compte en valeur de la colonne de tri (-7 = valeur $\geq$ courante - 7).

Fonctions de rang — rank, dense_rank, row_number

Trois fonctions similaires avec des comportements différents face aux ex-aequo

```
fenetre = Window.partitionBy("ville").orderBy(F.col("montant").desc())

df.select(

  "nom", "ville", "montant",
  F.rank()      .over(fenetre).alias("rang"),      # ex-aequo → saut de rang
  F.dense_rank().over(fenetre).alias("rang_dense"), # ex-aequo → pas de saut
  F.row_number().over(fenetre).alias("num_ligne"),  # ordre strict, pas d'ex-aequo
  F.percent_rank().over(fenetre).alias("rang_pct"), # [0,1]
  F.ntile(4)    .over(fenetre).alias("quartile"),  # découpe en N groupes
).show()
```

Comportement face aux ex-aequo — exemple avec montants [3000, 2200, 2200, 1500, 800] :

Fonction	Résultat	Comportement ex-aequo
rank()	[1, 2, 2, 4, 5]	Saut de rang après l'ex-aequo (pas de rang 3)
dense_rank()	[1, 2, 2, 3, 4]	Pas de saut — le rang suivant est toujours +1
row_number()	[1, 2, 3, 4, 5]	Ordre strict — les ex-aequo reçoivent des rangs différents arbitrairement

lag() / lead() — Accès aux lignes voisines

Accéder à la ligne précédente ou suivante dans la fenêtre — indispensable pour les analyses temporelles

```
w_temps = Window.partitionBy("region").orderBy("date")

df.withColumn("ca_precedent", F.lag("ca", 1).over(w_temps))\
  .withColumn(
    "ca_suivant", F.lead("ca", 1).over(w_temps))\
  .withColumn(
    "evolution_pct", F.round(
      (F.col("ca")-F.col("ca_precedent"))/F.col("ca_precedent")*100, 1))\
  .select(
    "region", "date", "ca", "ca_precedent", "evolution_pct").show()
```

Cumul progressif et moyenne glissante

```
w_cumul  = Window.partitionBy("region").orderBy("date").rowsBetween(Window.unboundedPreceding,
Window.currentRow)
w_gliss  = Window.partitionBy("region").orderBy("date").rowsBetween(-2, 0)  # 3 dernières lignes

df.withColumn("ca_cumule", F.sum("ca").over(w_cumul))\
  .withColumn("moy_3_mois", F.round(F.avg("ca").over(w_gliss), 2))
```


5. Les jointures (joins)

Les 7 types de jointures Spark

" inner "	Lignes présentes dans LES DEUX tables
" left "	Toutes les lignes gauche + correspondances droite (null sinon)
" right "	Toutes les lignes droite + correspondances gauche
" outer "	Toutes les lignes des deux côtés
" left_anti "	Lignes gauche SANS correspondance à droite ← clients sans commande
" left_semi "	Lignes gauche AVEC correspondance (sans colonnes droite)
" cross "	Produit cartésien — TOUTES les combinaisons

Syntaxe : `df1.join(df2, df1['id'] == df2['client_id'], 'inner')` — ou, si même nom de colonne :
`df1.join(df2, on='id', how='inner')`

Broadcast Join et colonnes ambiguës

Broadcast : évite le shuffle total en diffusant la petite table à tous les Executors

```
from pyspark.sql.functions import broadcast

# Sans broadcast : shuffle des DEUX tables ❌
df_large.join(df_reference, "code_produit")

# Avec broadcast : df_reference copié en mémoire sur chaque Executor ✅
df_large.join(broadcast(df_reference), "code_produit")

spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "10mb") # seuil automatique (10 Mo par défaut)
```


Broadcast Join et colonnes ambiguës

Éviter les colonnes ambiguës après jointure

```
# ❌ Problème : deux colonnes 'id' après jointure → AnalysisException  
df_clients.join(df_commandes, df_clients["id"] == df_commandes["client_id"])
```

```
# ✅ Solution 1 : même nom de colonne → colonne unique dans le résultat  
df_c = df_clients.withColumnRenamed("id", "client_id")  
df_c.join(df_commandes,  
"client_id")
```

```
# ✅ Solution 2 : alias de DataFrame  
df_clients.alias("c").join(df_commandes.alias("o"), F.col("c.id")==F.col("o.client_id"))\  
.select(F.col(  
"c.id"), "nom", "montant")
```

```
# ✅ Solution 3 : drop() après jointure  
df_joint.drop(df_commandes["client_id"])
```

Résumé du module

Concept	Points clés à retenir
Shuffle	Opération la plus coûteuse — redistribution réseau par clé — filtrer AVANT le groupBy
reduceByKey()	Réduction locale avant shuffle → beaucoup plus efficace que groupByKey()
groupByKey()	Envoie toutes les valeurs au réseau → à éviter sur gros volumes (risque OOM)
aggregateByKey()	Version la plus flexible — peut produire un type différent des valeurs d'entrée
groupBy().agg()	Agrégations multiples en une seule passe — toujours grouper les agg() ensemble
pivot()	Valeurs de lignes → colonnes — spécifier les valeurs explicitement pour éviter un scan
Window functions	Calculs sur une fenêtre sans réduire le DataFrame — rank, lag, cumul, glissement
rank() vs dense_rank()	rank() saute les rangs après ex-aequo, dense_rank() non, row_number() est strict
lag() / lead()	Accès à la ligne précédente/suivante — incontournable pour les analyses temporelles
Broadcast Join	Petite table (< 10 Mo) copiée sur chaque Executor — évite le shuffle de la grande table
Colonnes ambiguës	Renommer avant jointure, utiliser des alias de DF, ou drop() la colonne en double

Exercices

Ex. 1 — RDD Reducers (30 min) Sur un RDD de logs (ip, url, durée_ms) : nb requêtes par IP avec reduceByKey(), durée moyenne par URL avec aggregateByKey(), IP avec le plus de requêtes — comparer groupByKey() vs reduceByKey().

Ex. 2 — Agrégations DataFrame (25 min) Sur un jeu de ventes : statistiques complètes (min, max, moy, écart-type, médiane) par catégorie, tableau croisé pivot() catégorie × trimestre, identifier les catégories avec CA croissant chaque trimestre.

Ex. 3 — Window functions (35 min) Sur (id, nom, département, salaire, date_embauche) : rang par département/salaire, écart vs moyenne du département, salaire du collègue embauché juste avant (lag), salaire cumulé par département.

Ex. 4 — Jointures et optimisation (30 min) Jointure triple clients + commandes + produits. Anti-join clients sans commande. Comparer plan d'exécution avec/sans broadcast() via .explain() — mesurer l'impact sur le temps d'exécution.

Pour aller plus loin

Documentation

- Window Functions SQL : spark.apache.org/docs/latest/sql-ref-syntax-qry-select-window.html
- Optimisation des jointures : spark.apache.org/docs/latest/sql-performance-tuning.html

Stratégies de jointure dans la Spark UI

BroadcastHashJoin

Petite table broadcastée — pas de shuffle sur la grande table

SortMergeJoin

Deux grandes tables — tri + merge après shuffle (cas le plus courant)

ShuffleHashJoin

Shuffle sur les deux tables, puis hash join — alternatives à SMJ

Astuce Spark UI : onglet SQL → cliquer sur la requête → examiner le plan physique. Repérer les nœuds **Exchange** (shuffles) et les stratégies de jointure pour identifier les goulots d'étranglement.

Module suivant → Module 6 : Partitionnement, shuffles et collecte — contrôler finement la distribution des données